

Engineering Prompt Templates

A standardized framework for development, debugging, and system optimization.

1. Starting a New Task

<code><role></code>	Defines seniority and domain expertise.
<code><context></code>	Background info and project requirements.
<code><goal></code>	The explicit outcome or feature to be delivered.
<code><constraints></code>	Technical stack, performance, or architectural limits.
<code><rules></code>	Clean code principles and implementation standards.
<code><output_format></code>	Structural expectations for the final result.

IMPLEMENTATION EXAMPLE

```
<role>
  Senior React Developer specializing in accessible Design Systems.
</role>

<context>
  Building a modular modal component for an enterprise dashboard.
</context>

<goal>
  Create a reusable modal with props for title, content, and footer actions.
</goal>

<constraints>
  TypeScript only, no external UI libraries. Must handle focus trapping.
</constraints>

<rules>
  Decouple styles from logic, maintain WAI-ARIA compliance.
</rules>

<output_format>
  Functional component with documented TypeScript interfaces.
</output_format>
```

2. Bug Resolving

<code><problem_statement></code>	High-level summary of the issue.
<code><expected_behavior></code>	The intended system state.
<code><actual_behavior></code>	Observed errors or logs.
<code><problem_scenario></code>	Reproduction steps and environment details.
<code><attempts></code>	Steps already taken to avoid repetition.
<code><goal></code>	Permanent resolution with zero regressions.

IMPLEMENTATION EXAMPLE

```
<role>
  Senior Node.js Backend Engineer with expertise in stream processing.
</role>

<problem_statement>
  API returns 500 error on file uploads exceeding 5MB.
</problem_statement>

<expected_behavior>
  Successful upload to cloud storage and 201 Created response.
</expected_behavior>

<actual_behavior>
  Server crash due to memory limit reached during buffer allocation.
</actual_behavior>

<attempts>
  Increased timeout and memory limit; crash still occurs under load.
</attempts>

<goal>
  Implement stream-based upload handling to minimize memory footprint.
</goal>

<rules>
  Maintain backward compatibility for small files; avoid external middleware.
</rules>
```

3. Enhancement

<role>	Optimization or architectural specialist.
<context>	Current system metrics or performance bottlenecks.
<goal>	Specific target improvement.
<success_criteria>	Measurable KPIs (e.g., latency, code coverage).
<constraints>	Boundaries (e.g., no DB migrations).

IMPLEMENTATION EXAMPLE

```
<role>
  Full-stack Performance Engineer....
</role>

<context>
  Chat widget TTI is currently 4.2 seconds.
</context>

<goal>
  Optimize initial load sequence.
</goal>

<success_criteria>
  Reduce TTI by >30%.
</success_criteria>

<constraints>
  Cannot change the widget provider.
</constraints>

<rules>
  Implement lazy loading and tree shaking.
</rules>
```